Systemd

¿Que es systemd?

Sistema y administrador de servicios para Linux, compatible con scripts de inicio SysV. *Systemd* hace uso de la paralelización agresiva, socket y activación *D-Bus* para iniciar los servicios. Ofrece la puesta en marcha de demonios bajo demanda, realiza el seguimiento de procesos utilizando *Linux cgroups*, soporta copia instantánea de volumen y la restauración de estado del sistema, mantiene puntos de montaje y automontaje e implementa un elaborado servicio lógico de control transaccional basado en la dependencia.

Vamos a desglosar un poco esta definición:

- **Capacidades de paralelización agresiva usando socket**: systemd crea de una misma vez todos los *sockets* para todos los demonios acelerando así el arranque completo e iniciar más procesos en paralelo. En un segundo paso *systemd* ejecutará a la vez todos los demonios.
- **Activación D-Bus para iniciar servicios**: Utilizando la activación D-Bus, un servicio puede ser iniciado la primera vez que es accedido.
- **Seguimiento de procesos utilizando Linux cgroups**: cgroup también llamados grupos de control, es una característica del kernel para crear límites, políticas e incluso explicar el uso de los recursos de ciertos grupos de procesos. cgroup asocia un conjunto de tareas con un conjunto de parámetros, para uno o más subsistemas, proporcionando un control de servicios y tareas, así como todos sus futuros 'hijos' en grupos jerárquico. Un subsistema es un módulo resultado de la agrupación de diversas tareas con el fin de mantener un mejor control sobre estas de forma particular.

Nota: La herramienta **systemd-cgls** nos muestra recursivamente el contenido del árbol de jerarquías de un determinado grupo de control de Linux.

- **Mantiene puntos de montaje y automontaje**: Puede utilizarse para montar o desmontar los puntos de montaje, quedando */etc/fstab* como una fuente de configuración adicional a la que podremos llamar para su supervisión con la opción "comment=" de fstab para marcar las entradas controladas por systemd.

En definitiva, **systemd** ha sido creado para ofrecer un inicio mas rápido y flexible que SysV, permitiendo el arranque paralelo de servicios y su inicio basado en la detección de conexión de nueva unidad externa.

Nota: Hasta ahora el **PID1** era para el programa **init**, cosa que ha cambiado en systemd a favor de **/usr/lib/systemd/systemd** y además *systemd* al igual que *Upstart* deja de utilizar el archivo */etc/inittab*

Algunas de las mejoras que ofrece systemd:

- Se ha mejorado sustancialmente la velocidad de inicialización del sistema
- systemd asume que cualquier dispositivo puede ser conectado o desconectado en cualquier momento (**hotplug**)
- systemd utiliza la activación de daemons por medio de sockets, aportando capacidades de paralelización
- Una de sus características es el registro (**journal**) mediante **cgroups** de todos los servicios y procesos iniciados
- systemd es *modular*, esto quiere decir que se han elaborado una seríe de "paquetes" en los que varios servicios son administrados de forma conjunta

¿Que son las Units / Unidades de Servicios?

systemd inicia y supervisa todo el sistema y se basa en la noción de unidades, compuestas de un nombre (el nombre del demonio) y una extensión. Será la extensión la que indique de que tipo de unidad se trata. Además cada unidad tiene su correspondiente archivo de configuración cuyo nombre es idéntico. Un ejemplo sería el servicio **httpd.service** cuyo archivo de configuración sería *httpd.service*. Los archivos de unidades disponibles en nuestro sistema podemos encontrarlos en **/usr/lib/systemd/system/** y **/etc/systemd/system/**

Nota: Los archivos bajo el directorio */etc/systemd/system/* prevalecerán en caso de duplicados.

Existen siete tipos diferentes de unidades:

- **service**: Demonios que pueden ser iniciados, detenidos, reiniciados o recargados.
- **socket**: Esta unidad encapsula un socket en el sistema de archivos o en Internet. Cada unidad socket tiene una unidad de servicio correspondiente.
- **device**: Esta unidad encapsula un dispositivo en el árbol de dispositivos de Linux.
- **mount**: Esta unidad encapsula un punto de montaje en la jerarquía del sistema de archivos.
- **automount**: Encapsula un punto de montaje automático. Cada unidad *automount* tiene una unidad *mount* correspondiente, que se inicia al acceder al directorio de automontaje.
- **target**: Utilizada para la agrupación lógica de unidades. Referencia a otras unidades, que pueden ser controladas conjuntamente, un ejemplo sería **multi-user.target**, que básicamente desempeña el papel de nivel de ejecución 3 en el sistema clásico SysV.
- **snapshot**: Similar a las unidades target.

Entonces los archivos de configuración tendrán los nombres: programa.*service*, socket.*socket*, *dispositivo.device*, *puntodemontaje.mount*, etc…

Nota: La unidad **default.target** activa servicios y otras unidades dependientes, durante el arranque de systemd. Esto lo comentaremos en el apartado *target*

Compatibilidad de systemd con SysV

systemd al igual que *Upstart* ofrece compatibilidad con *SysV* (comando *service* y *chkconfig*) para aquellos servicios que aun soportan o funcionan únicamente con scripts de inicio *SysV* (actualmente en 2015, son pocos los servicios '*comunes*' que corren bajo *SysV*). *Upstart* pese a mantener compatibilidad con los comandos *service* y *chkconfig* de *SysV* implementó su propia utilidad para la administración de servicios, de igual modo *systemd* lo hace con su herramienta **systemctl**

#systemctl stop nombreservicio.service

En *SysV* habilitábamos servicios con *chkconfig* (o reproducíamos listas de estos para ver cual de ellos se ejecutaba al arranque), algo que ahora bajo *systemd* podemos hacer con los siguientes comandos:

Habilitar el servicio httpd al arranque del sistema (*chkconfig httpd on* , para SysV):

#systemctl enabled httpd.service

Listar todas las unidades de servicios instaladas (algo parecido a *chkconfig –list*)

#systemctl list-unit-files

O solo aquellas que se encuentran en activadas:

#systemctl list-units o **#systemctl**

Podemos apreciar que a la hora de pasar el nombre del servicio lo hacemos con el nombre completo, es decir incluyendo su sufijo, pero existen una serie de atajos:

- Si no se especifica el sufijo, *systemctl* asumirá que es .service. Por ejemplo, **netcfg** y **netcfg.service** se consideran equivalentes.
- Los puntos de montaje se traducirán automáticamente en la correspondiente unidad *.mount*. Por ejemplo, si especifica **/home** será equivalente a **home.mount**.
- Similar a los puntos de montaje, los dispositivos se traducen automáticamente en la correspondiente unidad *.device*, por lo tanto, la especificación **/dev/sda2** es equivalente a **dev-sda2.device**.

Nota: Consulte man *systemd.unit* para más detalles.

Uso de systemctl

La utilidad de administración de las unit de systemd es **systemctl**, la cual combina las herramientas *service* y *chkconfig* de SysV, por lo tanto podremos arrancar, parar, recargar servicios, activar o desactivar servicios en el arranque, listar los estados de los servicios,etc…

Creo que con la siguiente tabla veremos el uso básico de *systemctl* además de sus "iguales" en SysV.

Acción	systemd	SysV
Arrancar un servicio	systemctl start foo	service foo start
Detener un servicio	systemctl stop foo	service foo stop
Reiniciar un servicio	systemctl restart foo	service foo restart
Recargar el archivo de configuración de un servicio (en systemd no todos los servicios lo soportan)	systemctl reload foo	service foo reload
Rearrancar un servicio que ya se encuentra en ejecución	systemctl condrestart foo	service foo condrestart
Mostrar el **estado** del servicio	systemctl status foo	service foo status
Activar un servicio para que sea ejecutado durante el arranque	systemctl enable foo	chkconfig foo on
Desactivar un servicio para que no sea ejecutado durante el arranque	systemctl disable foo	chkconfig foo off
Muestra el **estado** de un servicio durante el **arranque**	systemctl is-enable foo (1*)	chkconfig --list foo
Crear o **modificar** el **archivo** de **configuración** de un servicio	systemctl daemon-reload	chkconfig --add foo
Listar los **modos de ejecución** para los que un servicio está activado o desactivado	ls /etc/systemd/system/*.wants/foo.service	chkconfig

1* Puede que un servicio este "**enable**" pero no tiene porqué estar activo cuando iniciemos sesión ya que puede que ese servicio este configurado para ejecutarse solo en determinados *runlevels* (o target en nuestro caso, veremos que son los target a continuación), a diferencia de *chkconfig –list de SysV* que mostraba todos los servicios con todos los *runlevels* posibles y para cada uno indicaba si estaba on o off. Para conseguir algo parecido en *systemd*, tendríamos que listar los **target** disponibles y veríamos que servicios se ubican dentro de estos, así sabremos con que *target* (o runlevel) un servicio será iniciado. Podemos emplear el siguiente listado

```
#ls /etc/systemd/system/*.wants/httpd.service
/etc/systemd/system/multi-user.target.wants/httpd.service
```

Este comando nos devuelve que el servicio *httpd* se encuentra bajo *multi-user.target* lo que viene siendo el runlevel 3 de SysV. En los siguientes apartados conoceremos los distintos *target* que de algún modo tienen similitud con los *runlevels* de SysV

Nota: Podríamos haber omitido el subdirectorio */httpd.service* para conocer la lista completa de *targets* y sus unit (mount, service, socket…)

Podemos hacerlo de una forma inversa (para lo cual hay que conocer el target), es decir ver que servicios están ejecutándose para un target concreto:

#systemctl show -p "Wants" multi-user.target

Además de los comandos de la tabla también podemos ver las *units* de servicios que tenemos cargados en el sistema con el comando:

#systemctl -t service list-units –all

Con este comando veremos que servicios están cargados y además si están activos o muertos a parte de una pequeña descripción.

Nota: Podemos cambiar service por mount, socket, device… para listar otros tipos (**-t**) de units.

Si en vez de ver las units cargadas queremos ver cuantas hay instaladas (es decir aquellas que tienen archivos de configuración instalados en nuestro sistema y por lo tanto están disponibles):

#systemctl -t service list-unit-files –all

Nota: Podemos cambiar el tipo de unit o directamente omitir el parámetro **-t** y listar todas.

Sugerencia: Puede utilizar las siguientes órdenes **systemctl** con el parámetro **-H usario@host** para controlar una instancia de *systemd* en una máquina remota. Esto utilizará SSH para conectarse a la instancia *systemd* remota.

Target

systemd utiliza *target* en vez de runlevels (0123456) que reciben un nombre (en vez de un número) para identificar un propósito específico, con la posibilidad de realizar más de una acción al mismo tiempo. Algunos *targets* pueden heredar todos los servicios de otro *target* e implementarse con servicios adicionales. La siguiente tabla muestra la similitud entre algunos de los target con los runlevels de SysV:

Nivel de ejecución sysvinit	Target systemd	Notas
0	runlevel0.target, poweroff.target	Detener el sistema.
1, s, single	runlevel1.target, rescue.target	Modo monousuario.
3	runlevel3.target, multi-user.target	Multiusuario, no gráfico. Los usuarios pueden usualmente hacer login por medio de múltiples consolas o desde la red.
5	runlevel5.target, graphical.target	Multiusuario, gráfico. Usualmente todos los servicios del nivel de ejecución 3 sumando el login gráfico.
6	runlevel6.target, reboot.target	Reiniciar el sistema.
emergency	emergency.target	Shell de emergencia.

Nota: Actualmente no existen *target* similares a los runlevels 2 y 4 de SysV, pero podríamos definirlos nosotros mismos.

Existen otros target que podremos ver con el comando:

#systemctl list-units –type=target

Podemos cambiar de *target* (o modo de ejecución) actual con el comando:

 # systemctl isolate graphical.target

Nota: Esto podría ser equivalente a la orden *#telinit 5 de SysV*

systemd también nos permite cambiar el *target* predeterminado e incluso añadir nuevos servicios a otros *target*, pero antes de esto, es importante dejar claro algunos de los directorios de los que hace uso *systemd*:

- Los archivos **unit** (archivos de configuración de service, mount, device…) se encuentran en: **/usr/lib/systemd/system/** o **/etc/systemd/system/**
- Los *target* (runlevels) igualmente pueden situarse en ambos directorios
- Los directorios ***.wants** (ubicados igualmente en ambos directorios) contienen los enlaces simbólicos que apuntan a determinados servicios, serán estos servicios los que se ejecuten con el *target* al que corresponde dicho directorio *wants*, recordar que si un *target* precisa de otro, también serán cargados los servicios de este otro *target*.

Nota: Los archivos unit de */etc/systemd/system* tienen una mayor precedencia sobre archivos unidad de */lib/systemd/system*

¿Quieres cambiar el target (runlevel) predeterminado de arranque?

El *target* ***/etc/systemd/system/default.target*** es el *target* predeterminado de arranque, es un enlace simbólico que por defecto apunta a **/lib/systemd/system/graphical.target** por lo que para cambiar de *target* de arranque, bastará con eliminar dicho link y crear uno nuevo apuntando al nuevo *target*.

Si quisiéramos podríamos crear un target desde 0 a nuestro gusto y darle un nombre (*mylevel.target*) y a continuación habilitarlo:

 # systemctl enable mylevel.target

El efecto de esta orden crea un enlace simbólico (*/etc/systemd/system/default.target*) que apunta a nuestro *mylevel.target*. Esto solo funciona si hemos añadido la etiqueta **[Install]** al archivo, de la siguiente manera:

[Install]
Alias=default.target

También podemos añadir o quitar nuevos servicios a un *target* determinado, bastará con crear nuevos enlaces simbólicos dentro del directorio **.wants* del *target* de arranque (o de algunos de los que dependa) apuntando a los servicios deseados.

Otra forma de modificar el *target* de inicio es a través de los parámetros que le pasamos al kernel en el archivo de configuración del gestor de arranque añadiendo por ejemplo **systemd.unit=multi-user.target** para arrancar en nivel3

Journal

journal es un componente mas de *systemd*, que capta los mensajes *syslog*, los mensajes de registro del kernel, los del disco RAM inicial y los mensajes de arranque iniciales, así como mensajes escritos en *stdout* y *stderr* de todos los servicios, haciendo que estén disponible para el usuario. Se puede utilizar en paralelo, o en lugar de un demonio *syslog* tradicional, como *rsyslog* o *syslog-ng*.

Hasta Fedora 18 (e inclusive a día de hoy, Enero del 2015, en Centos7) el registro de *journal* se almacena de forma volátil en **/run/log/journal** por lo que será eliminado tras un reinicio. La intención es que venga configurado por defecto de manera que sea un registro persistente y con directorio principal **/var/log/journal** (que no venga por defecto no quiere decir que no lo podamos configurar de tal modo).

Una de las razones por las que *syslog* comenzaba a quedarse "anticuado" es la necesidad por ejemplo de mostrar las últimas líneas del registro de un determinado servicio en el momento de preguntar por su estado (con *systemctl*), esto necesita que archivos de registro sean descomprimido en tiempo real y a la vez mostrado, algo que con *syslog* se hacía casi imposible y además, ineficiente y poco seguro.

Hasta ahora nos haremos la idea de que journal es el sustituto de *syslog*, el cual nos permite filtrar la salida del registro por campos, dejar el registro de forma volátil o volverlo persistente mediante su archivo de configuración (**/etc/systemd/journald.conf**), darle un tamaño máximo al fichero de registro, permite la compatibilidad con el antiguo *syslog* mediante el *socket /run/systemd/journal/syslog* (tendremos que asociar *syslog* a este *socket* y no a */dev/log*, el paquete *syslog-ng* proporciona automáticamente la configuración necesaria), reenviar la salida del registro a una terminal, etc…

COMANDO SYSTEMCTL

systemctl: Nos permite controlar el demonio *systemd* y con ello llevar a cabo una eficaz administración de los servicios.

systemctl [opciones] comando [opciones del comando]

<u>Opciones:</u>

- Indicar el tipo de target: **-t** (–type=TARGET)
- Filtrar la salida en función del estado: **–state=**LOAD|SUB|ACTIVE
- Filtrar por las propiedades de unidad/servicio/trabajo: **-p** (–property=)
- Mostrar todas las unidades/servicios/job cargados: **-a** (–all)
- Si filtramos por socket, podemos indicar de que tipo de socket se trata: **–show-types**
- Si vamos a habilitar una unidad, podemos sobreescribir su enlace: **-f**

Comandos:

- Listar las unidades conocidas (opción por defecto): **list-units**
- Listar las unidades instaladas: **list-unit-files**
- Listar sockets: **list-sockets**
- Iniciar, Detener, Recargar (tipo *smb.conf*), Reiniciar y Estado de los servicios: **start**, **stop**, **reload** y **restar** (respectivamente)
- Inicia la unit indicada (y sus dependencias) y detiene el resto. Esto es parecido a cambiar de runlevel: **isolate**
- Entrar al modo por defecto (parecido a isolate pero sin especificar el target): **default**
- Entrar en modo de rescate: **rescue**
- Entrar en modo de emergencia: **emergency**
- Cerrar la sesión y apagar el sistema: **halt**
- Apagar el sistema (como si quitamos el cable de corriente): **poweroff**
- Reiniciar el sistema: **reboot**
- Entrar en modo suspensión: **suspend**
- Hibernar el equipo: **hibernate**
- Comprobar si una unit se encuentra en ejecución: **is-active**
- Comprobar si una unit está en estado "failed": **is-failed**
- Mostrar las propiedades de una unit: **show**
- Mostrar las dependencias de una unit (si no especificamos una en concreto se toma default.target): **list-dependencies**
- Habilitar / Deshabilitar una unit: **enable** , **disable**
- Comprobar si una unidad está habilitada: **is-enabled**
- Resetear una unit: **preset**
- Mostrar el runlevel (target) que tenemos por defecto para iniciar el sistema: **get-default**
- Setear el target con el que queremos arrancar el sistema: **set-default**
- Listar los trabajos (Jobs) que están activos: **list-jobs**
- Cancelar un trabajo: **cancel**
- Recargar el archivo de una unidad (httpd.service): **daemon-reload**

shutdown, **poweroff**, **halt**, **reboot**: Estos comandos son equivalentes a **init 0** e **init 6**. La mejor forma de apagar el sistema es con *shutdown*. Además este permite una gran variedad de opciones.

shutdown [opcion] argumento [mensaje]

Opciones:

- Detener el sistema: **-H**
- Apagar el sistema: **-P**
- Apagar el sistema (equivalente a -P a menos que -H haya sido especificada): **-h**
- Reiniciar el sistema: **-r**
- Cancelar una tarea de shutdown pendiente: **-c**
- No enviar mensajes después de ejecutar la orden shutdown: **–no-wall**

Ejemplos:

- Apagar el sistema a las 01:30 AM y además envía un mensaje

```
# shutdown -h 1:30 "A las 1:30 de la madrugada el sistema va a ser apagado"
```

- Cancelamos la acción anterior, pues nos hemos equivocado de hora:

```
# shutdown -c "El sistema no va a ser apagado, quizás a las 13:30, pero avisaremos. Disculpen"
```

Laboratorio SystemD.

Introducción.

Systemd es un nuevo sistema para la administración de dispositivos, eventos y servicios en GNU/Linux creado por Lennart Poettering. Es el reemplazo para **sysvinit**, **upstart** y **udev** en la mayoría de las distribuciones modernas. Se utiliza en **CentOS 7**, **Red Hat™ Enterprise Linux 7** y versiones recientes de prácticamente todas las distribuciones de GNU/Linux, incluyendo **Debian**, **Ubuntu™** y **Fedora™**.

A pesar de tratarse de tecnología de vanguardia, es compatible con los métodos utilizados en el pasado en **SysV** y **LSB** y las mejoras respecto de éstos incluyen capacidades de activación de zócalos y buses que permiten una mejor ejecución en paralelo de servicios independientes y el uso de **cgroups** para realizar el seguimiento de los procesos del servicio en lugar de utilizar PIDs. Ésto último impide que los servicios puedan evadir la administración de systemd.

Procedimientos.

Niveles de ejecución.

Los niveles de ejecución utilizados en SysV siguen siendo los mismos. Sin embargo en SystemD se administran como *objetivos* o *targets*. La siguiente tabla muestra los niveles de ejecución de SysV y su equivalente en SystemD:

SysV	SystemD	Uso
0:	runlevel0.target o poweroff.target	Apaga el sistema
1 o S:	runlevel1.target o rescue.target	Nivel mono-usuario
2:	runlevel2.target o multi-user.target	Multi-usuario. Idéntico al nivel 3
3:	runlevel3.target o multi-user.target	Multi-usuario
4:	runlevel4.target o multi-user.target	Multi-usuario. Idéntico al nivel 3
5:	runlevel5.target o graphical.target	Multi-usuario con servidor de video
6:	runlevel6.target o reboot.target	Reinicia sistema
emergency	emergency.target	Intérprete de mandatos de emergencia.

Determinar el nivel de ejecución actual.

Si antes ejecutaba **runlevel** o **who** con la opción **-r** para determinar el nivel de ejecución actual, éstos siguen funcionado, sin embargo se prefiere se utilice en su lugar lo siguiente:

```
systemctl list-units --type=target
```

Conmutar el nivel de ejecución.

Si antes usted utilizaba:

```
init 3
```

Lo anterior sigue funcionando, pero ahora se recomienda utilizar:

```
systemctl isolate multi-user.target
```

O bien:

```
systemctl isolate runlevel3.target
```

Si antes usted utilizaba:

```
init 5
```

Lo anterior sigue funcionando, pero ahora se recomienda utilizar:

```
systemctl isolate graphical.target
```

O bien:

```
systemctl isolate runlevel5.target
```

Cambiar el nivel de ejecución.

Si antes editaba el archivo /etc/inittab y establecía **id:3:initdefault:** para definir el nivel de ejecución 3, ahora debe ejecutar lo siguiente:

```
ln -sf /lib/systemd/system/graphical.target \
    /etc/systemd/system/default.target
```

Si antes editaba el archivo /etc/inittab y establecía **id:5:initdefault:** para definir el nivel de ejecución 5, ahora se debe ejecutar lo siguiente:

```
ln -sf /lib/systemd/system/multi-user.target \
    /etc/systemd/system/default.target
```

Gestión de servicios.

Si antes usted utilizaba:

```
chkconfig --list
```

Ahora debe utilizar:

```
systemctl list-unit-files
```

Si antes usted utilizaba:

```
chkconfig --list |grep "5:activo"
```

Ahora debe utilizar:

```
systemctl list-units
```

Si antes usted utilizaba:

```
chkconfig --list httpd
```

Ahora debe utilizar:

```
systemctl is-enabled httpd
```

Si antes usted utilizaba:

```
chkconfig httpd on
```

Ahora debe utilizar:

```
systemctl enable httpd
```

Si antes usted utilizaba:

```
chkconfig httpd off
```

Ahora debe utilizar:

```
 systemctl disable httpd
```

Si antes usted utilizaba:

```
service httpd start
```

O bien:

```
/etc/init.d/httpd start
```

Ahora debe utilizar:

```
systemctl start httpd
```

Si antes usted utilizaba:

```
service httpd restart
```

O bien:

```
/etc/init.d/httpd restart
```

Ahora debe utilizar:

```
systemctl restart httpd
```

Si antes usted utilizaba:

```
service httpd stop
```

O bien:

```
/etc/init.d/httpd stop
```

Ahora debe utilizar:

```
systemctl stop httpd
```

Si antes usted utilizaba:

```
service httpd reload
```

O bien:

```
/etc/init.d/httpd reload
```

Ahora debe utilizar:

```
systemctl reload httpd
```

Si antes usted utilizaba:

```
service httpd status
```

O bien:

```
/etc/init.d/httpd status
```

Ahora debe utilizar:

```
systemctl status httpd
```

Para ver los servicios que fallan:

systemctl --failed --type=service

Para ver los servicios habilidatos y desabilitados para todas las units:

systemctl list-unit-files --type=service --all

systemctl list-unit-files --type=service |grep -i httpd

Resumen Systemctl

Si antes usabas:
```
chkconfig --list
```

Ahora debes usar:
```
systemctl list-unit-files
```

Si antes usabas:
```
chkconfig --list |grep "5:activo"
```

Ahora debes usar:
```
systemctl list-units
```

Si antes usabas:
```
chkconfig --list httpd
```

Ahora debes usar:
```
systemctl is-enabled httpd
```

Si antes usabas:
```
chkconfig httpd on
```

Ahora debes usar:
```
systemctl enable httpd
```

Si antes usabas:
```
chkconfig httpd off
```

Ahora debes usar:
```
systemctl disable httpd
```

Si antes usabas:
```
service httpd start
```
o
```
/etc/init.d/httpd start
```

Ahora debes usar:
```
systemctl start httpd
```

Si antes usabas:
```
service httpd restart
```
o
```
/etc/init.d/httpd restart
```

Ahora debes usar:
```
systemctl restart httpd
```

Si antes usabas:
`service httpd stop`
o
`/etc/init.d/httpd stopt`

Ahora debes usar:
`systemctl stop httpd`

Si antes usabas:
`service httpd reload`
o
`/etc/init.d/httpd reload`

Ahora debes usar:
`systemctl reload httpd`

Si antes usabas:
`service httpd status`
o
`/etc/init.d/httpd status`

Ahora debes usar:
`systemctl status httpd`

Laboratorio agregar una aplicación a Systemd

En este laboratorio, agregaremos la aplicación supervisamem, como servicio a través de systemd.

Usted desea disponer en un nivel de ejecución determinado, de registros periódicos del consumo de memoria del servidor. Para ello creara el programa encargado de realizar dicho registro periódicamente, así como un script de inicio normalizado y que pueda utilizar **systemd**.

Cree en el directorio **/opt/scripts** con su editor favorito el archivo **supervisamen** con el siguiente contenido:

```
#!/bin/bash
while true
do
ahora=$(date "+%H:%M:%S - ")
echo -n $ahora >> /var/log/supervisamem.log
grep Dirty /proc/meminfo >> /var/log/supervisamem.log
sleep 30
done
```

Comandos Útiles

- chmod
- tail
- vi

Operaciones:

1. Hacer este archivo ejecutable
2. Crear en el directorio **/etc/init.d** un script normalizado de inicio de servicio para la aplicacion **supervisamen**
3. Hacer este archivo ejecutable
4. Comprobar el buen funcionamiento del programa ejecutando el servicio correspondiente.
5. Dejar que funcione unos minutos
6. Comprobar el contenido del archivo **/var/log/supervisamen.log**

 # cd /etc/systemd/system

 # vi supervisamem.service

[Unit]

Description=Daemon of axample

[Service]

ExecStart=/opt/scripts/iniciar-supervisamen start

ExecStop=/opt/scripts/iniciar-supervisamen stop

Type=forking

[Install]

WantedBy=multi-user.target

Tenemos que refrescar al systemd para que lea de nuevo las configuraciones, esto lo hacemos con systemctl daemon-reload

#systemctl daemon-reload

cd /opt/scripts/

[root@sercentos7 scripts]# ls

iniciar-supervisamen supervisamem

[root@sercentos7 scripts]# **systemctl start supervisamem**

[root@sercentos7 scripts]# **systemctl stop supervisamem**

Comprobaremos con el comando ps, y con el comando tail, si el servicio se inicia y se detiene correctamente.